

DECAPITULARE WARNING

Please set the font color to **green** if the answer is **confirmed**, otherwise leave it black. **(or red if it's wrong)**

QUIZ 1 + MIX

1. The following predicate is assumed to check if a tree is BST (Binary Search Tree):

```
is_BST(nil).  
is_BST(t(t(LL,KL,LR),K,t(RL,KR,RR))):-  
    KL<K<KR,!,  
    is_BST(t(LL,KL,LR)),  
    is_BST(t(RL,KR,RR)).
```

Choose the most specific (and correct) answer from below:

Select one:

- ☐ a. the predicate is incorrect as the inequality must be checked only after the 2 recursive calls
- ☐ b. the predicate is correct
- ☐ c. the predicate is incorrect, as the inequality check should be non-strict
- ☒ d. the predicate is incorrect, as the inequality check should consider other keys

it should check for these keys: $\text{maxKey_Left} < K, K < \text{minKey_Right}$
 $KL < K < KR$ isn't allowed in prolog, when it gets to a leaf it will fail

2. The following predicate should search for the first occurrence of an element in an incomplete list.

```
member(X, L):- var(L),!,fail.  
member(X, [_|_]):-!.  
member(X, [_|T]):-member(X,T).
```

Select one:

- ☐ a. is not correct, because it never finds the searched element
- ☐ b. is not correct, because it is deterministic
- ☐ c. is not correct, because it fails when reaching the end of the list
- ☒ d. is correct, because the clause for reaching the variable at the end of the list is the first, and it contains definitive failure

3. Which of the answers from below is correct and the most specific?

The list **[A,_]** is ...

Select one:

- ☒ a. a regular, complete list, as its variable termination _ does not occur after the pattern |
- ☐ b. a list with just one element (A) with an unknown value
- ☐ c. an incomplete list as A is a free variable
- ☐ d. an incomplete list as it contains _

4. Which of the answers from below is correct and the most specific ?

The list **[A,_]** may consist of : B)

Select one:

- ☐ a. at least 2 elements
- ☒ b. exactly 2 elements
- ☐ c. at least 1 element
- ☐ d. exactly 1 element

[A, _] -> length = 2

[A|_] -> length >= 1

5. Which of the answers from below is correct and the most specific?

The list [A|_] is ...

Select one:

- ☐ a. an incomplete list as it contains _
- ☐ b. a list with just one element (A) with an unknown value
- ☐ c. an incomplete list as A is a free variable
- ☒ d. an incomplete list as its variable termination _ occurs after the pattern |.

6. The following matching: [A,B|C] with [1,b] ends with:

Select one:

- ☐ a.
Failure as the first list has 3 elements while the second only 2 elements
- ☐ b.
Failure as letters and numbers wouldn't match
- ☐ c. Failure as A=1, B=b but C cannot unify
- ☒ d.
Success

7. In a search operation in an incomplete structure the following condition must hold true:

Select one:

- ☒ a. have explicit failure condition(s) in the beginning immediately followed by success condition(s)
- ☐ b. have explicit failure condition(s) or success condition(s)
- ☐ c. have explicit success condition(s) in the beginning immediately followed by explicit failure condition(s)
- ☐ d. have explicit failure condition(s) and success condition(s)

8. Which of the following assert sequences **CANNOT** generate the given clause order for the dynamic predicate insect/1:

```
insect(ant).
insect(bee).
insect(moth).
insect(fly).
insect(mosquito).
```

Select one:

- ☐ a. `asserta(insect(moth)) -> asserta(insect(bee)) -> assertz(insect(fly)) -> assert(insect(mosquito)) -> asserta(insect(ant))`
- ☒ b. `asserta(insect(mosquito)) -> asserta(insect(fly)) -> asserta(insect(moth)) -> assert(insect(bee)) -> asserta(insect(ant))`
- ☐ c. `asserta(insect(bee)) -> assert(insect(moth)) -> asserta(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))`
- ☐ d. `assert(insect(moth)) -> asserta(insect(bee)) -> asserta(insect(ant)) -> assert(insect(fly)) -> assert(insect(mosquito))`

a. :- dynamic insect/1.

insert):-

```
retractall(insect(_)),
asserta(insect(moth)),
asserta(insect(bee)),
assertz(insect(fly)),
assert(insect(mosquito)),
asserta(insect(ant)).
```

write):-

```
insect(R),
write(R).
```

9. The following implementation of the member predicate (for complete lists) does not behave correctly on incomplete lists, because:

```
member(X, [X|_]) :- !.
member(X, [_|T]) :- member(X, T).
```

Select one:

- ☐ a. it is deterministic
- ☐ b. it does not have a clause for when the input list becomes []
- ☐ c. it never finds the searched element
- ☒ d. it should fail explicitly when the search reaches the variable at the end of the list

10. In a search operation in an incomplete structure the following condition must hold true:

Select one:

- ☐ a. have a clause to check when reached a final variable: `fail,!,var(V)`.
- ☐ b. have a clause to check when reached a final variable: `var(V),fail,!.`
- ☒ c. have a clause to check when reached a final variable: `var(V),!,fail.`
- ☐ d. have a clause to check when reached a final variable: `!,var(V),fail.`

11. I want to write a predicate which replaces a given key in a BST with another key, specified as argument. The predicate should leave the tree unchanged if the searched key is not in the tree. Complete the partial implementation so that you get a correct predicate:

```
replace(_____,nil,nil).
replace(K,NK,t(K,L,R),t(NK,L,R)):-!.
replace(K,NK,t(Key,L,R),t(K_____,NL,R_____)):-
    K<Key,
    _____,
    replace(K,NK,L,NL).
replace(K,NK,t(Key,L,R),t(K,L,NR_____)):-
    replace(K,NK,R,NR).
```

replace K with Key in the third & fourth clause's result


```

replace_key(_, _, nil, nil).
replace_key(K, NK, t(K, L, R), t(NK, L, R)):-!.
replace_key(K, NK, t(Key, L, R), t(Key, NL, R)):-
    K < Key,
    !,
    replace_key(K, NK, L, NL).
replace_key(K, NK, t(Key, L, R), t(Key, L, NR)):-
    replace_key(K, NK, R, NR).

```

12. I want to write a predicate which finds the maximum sum on a branch of a BST. Complete the partial implementation so that you get a correct predicate:

```

maxSum(NIL, 0).
maxSum(t(K,L,R), Sum):-
    maxSum(L, SL),
    maxSum(R, SR),
    Sum is max(SL, SR) +K.

```

13. Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). **Use H whenever you need a NEW variable for an element from the list:**

```

one_pass([H1,H2|T], [H1|R]):- H1 =< H2, !, one_pass([H2|R], R).
one_pass([H1,H2|T], [H2|R]):-one_pass([H1|T], R).
one_pass([H1], [H1]).
one_pass([], []).

?- one_pass([3,1,4,5,2], R).
R=[1,3,4,2,5];
no.

```

in the first clause's recursive call replace [H2|R] with [H2|T]. BTW the last clause is not needed

```

one_pass([H1, H2|T], [H1|R]):-
    H1 =< H2, !,
    one_pass([H2|T], R).
one_pass([H1, H2|T], [H2|R]):-

```

```

one_pass([H1|T], R).
one_pass([H], [H]).
one_pass([], []).

```

14. Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). **Use H whenever you need a NEW variable for an element from the list:**

In the wrong fields, [H] should appear (in both)

```

one_pass([H1,H2|T], [R]✓):- H1=<H2, !, one_pass([T]✓, R).
one_pass([H1,H2|T], [H2,H1|F]✓):-one_pass([T]✓, R).
one_pass([R]✗, [R]✗).
one_pass([], []).

?- one_pass([3,1,4,5,2], R).
R=[1,3,4,2,5];
no.

```

15. Complete the predicate below such that it performs the merge operation on two ordered lists:

```

merge([H1|T1], [H2|T2], [H2|R]):-H1>H2, !, merge([H1|T1], T2, R).
merge([H|T1], T2, [H|R]):-merge(T1, T2, R). % use H as the first element in the list you need to apply [H|T] on
merge([], L, L).
merge(L, [], L).

?-merge([1,4,8], [2,5,8],L).
L=[1,2,4,5,8,8];
no

```

16. Which categories of nodes are frozen (prevented from backtracking) by a cut:

Select one:

- ☐ a. Any node to its left before the execution of the cut
- ☐ b. The parent node if the cut is the last subgoal
- ☒ c.
Its parent node after the execution of the cut
- ☐ d. Any sibling node to its right after the execution of the cut

17. The following matching: [1,2,3] with [A,B|C] would provide the following instantiations:

Select one:

- ☐ a. The matching fails
- ☒ b.
A=1, B=2, C=[3]
- ☐ c.
No instantiation is provided
- ☐ d. A=1, B=2, C=3

18. The predicate below is assumed to remove exactly one occurrence (in turn) of the first argument, from the list given in the second argument, to generate the list in the third argument. To behave accordingly (as in the associated example), the code should be updated in the following way:

THE CORRECT ANSWER IS G


```

delete(X, [X|T], T) .

delete(X, [H|T], [H|R]) :-
    delete(X, T, R) .

delete(_, [], []).

?- delete(b, [a,b,c,b,d], R) .

R= [a,c,b,d]? ;
R= [a,b,c,d]? ;
no

```

Select one:

- ☐ a. Swap the order of clauses 1 and 3.
- ☐ b. Update the clause 2 as `delete(X,[H|T], R):-!,delete(X,T,R).`
- ☒ c. Add another clause like: `delete(X,[X],[]).`
- ☐ d. Update the clause 2 as `delete(X,[H|T],[H|R]):-!,delete(X,T,R).`
- ☐ e. Update the clause 1 as `delete(X,[X|T],T):-!`.
- ☐ f. Remove the clause `delete(X,[X|T],T).`
- ☐ g. Remove the clause `delete(_,[],[]).`
- ☐ h. Swap the order of clauses 1 and 2.

19. Which of the answers from below is correct and the most specific?

The list `[]` is ...

Select one:

- ☒ a. not an incomplete list as it is missing the `|` pattern.
- ☐ b. not an incomplete list as there are no other elements in the list other than the final variable.
- ☐ c. an incomplete list.
- ☐ d. not an incomplete list as it contains only one single variable element.

20. Say we want to write a predicate which adds an element at the end of some incomplete list, given as argument. The predicate should:

21. The following sequence is assumed to be used in a condition-controlled loop (while or repeat-until) bubble sort for an input without duplicates. Would it be a sound

Select one:

- ☒ a. traverse the given list, to reach the variable at the end, then instantiate that variable accordingly
- ☐ b. traverse the given list, instantiate an additional result variable accordingly when reaching the end of the list, then copy all the input list elements in the result, as recursion returns, to maintain their order
- ☒ c. traverse the given list, and add the elements of the input list, one by one, in an additional result argument, as recursion proceeds, making sure to add the element at the end of the result list (when recursion ends)
- ☐ d. directly instantiate the variable at the end of the given list, accordingly (no traversal needed)

choice (that is, can be bubble sort implemented using it)?

```
traverse_once([A,B|T],[A|R]):-  
    A<B,!  
    traverse_once([B|T],R).  
traverse_once([A,B|T],[B|R]):-  
    traverse_once([A|T],R).  
traverse_once([A],[A]).
```

Select one:

- ☒ a. Yes! If properly employed, that is, until no swap occurred in the latest traverse_once.
- ☐ b. No! A bubble sort using it might enter an infinite loop.
- ☐ c. No! It (traverse_once) fails as it never reaches the empty list.
- ☐ d. Yes, but only if case traverse_once is called n times (where n is the length of the input).

22. If the code below uses append with the fact as the first clause, for the input

[4,7,3,5,9] what would be the second selection of A and B for swapping (setting B,A) during execution?

```
bubbleSort (In, Out) :-  
    append (X, [A, B|Y], In),  
    A > B, !,  
    append (X, [B, A|Y], Int),  
    bubbleSort (Int, Out).  
bubbleSort (In, In).
```

Select one:

- ☐ a. A=3, B=5
- ☐ b. A=4, B=7
- ☒ c. A=4, B=3
- ☐ d. A=7, B=5

```
bubbleSort(In ,Out) :-  
    append(X, [A,B|Y], In),  
    A > B,  
    !,  
    append(X, [B,A|Y], Int),  
    write(A),  
    write(" "),  
    write(B),  
    write("\n"),  
    bubbleSort(Int, Out).  
bubbleSort(In, In).
```

23. If the code below uses append with the fact as the first clause, for the input [3,6,2,4,1] what would be the second selection of A and B for swapping (setting B,A) during execution?

```

bubbleSort (In, Out) :-
    append(X, [A, B|Y], In),
    A > B, !,
    append(X, [B, A|Y], Int),
    bubbleSort (Int, Out).
bubbleSort (In, In).

```

Select one:

- ☒ a. A=3, B=2
- ☐ b. A=3, B=6
- ☐ c. A=2, B=4
- ☐ d. A=6, B=4

24. If we want a sorting by selection strategy (below), where min and delete are correct predicates to find the min value, and delete a value from a list respectively, which condition must hold true (choose the most specific one):

```

sort (In, [M|Out]) :-
    min (In, M),
    delete (M, In, Int),
    sort (Int, Out).
sort ([], []).

```

Select one:

- ☐ a. min needs to use a strict inequality
- ☒ b. delete needs to delete the first occurrence of the value from the list
- ☐ c. min needs to choose the first occurrence of the smallest value from the list
- ☐ d. min needs to be implemented with forward recursion

25. The predicate below should increasingly sort by selection, using max and delete

correct predicates to find the max value, and delete a value from a list, respectively.
 Fill in the blanks for a correct predicate:
 WHY YOU FUCKING WITH OUR MINDS POTOLEA

```

wrapper(In, Out) :-
    sort(In, [], Out).

sort(In, Acc, [M|Out]) :-
    max(In, M),
    delete(M, In, Int),
    NA = [Acc],
    sort(Int, NA, Out).

sort([], Out, Out).
  
```

wrapper(In, Out):-
 sorting(In, [], Out).

sorting(In, Acc, Out):-
 maximum_pretty(In, M),
 delete(M, In, Int), // it can be delete(In, M, Int). as well
 NA = [M|Acc],
 sorting(Int, NA, Out).
 sorting([], Out, Out).

26. The following code is assumed to count the internal nodes with 2 children of an incomplete binary tree. Choose the most specific answer from below:
 THE CORRECT ANSWER IS C


```

count_2_children(t(L,_,R),N):-var(L),!,
    count_2_children(R,N).
count_2_children(t(L,_,R),N):-var(R),!,
    count_2_children(L,N).
count_2_children(t(L,_,R),NN):-
    count_2_children(R,NR),
    count_2_children(L,NL),
    NN is NL+NR+1.
count_2_children(t(L,_,R),0):-
    var(L),var(R),!.

```

Select one:

- ☐ a. The predicate is correct.
- ☐ b. The predicate is incorrect, as is missing a clause on `var(T)`.
- ☐ c. The predicate is incorrect as the last clause should be the first one.
- ☐ d. The predicate is incorrect as the clause right subtree is processed before the left one.

27. The following sequence is assumed to search for a key in an incomplete BST.
Choose the most specific answer from below:

```

search(t(_,Key,_),Key):-!.
search(t(L,Key1,_),Key):-
    Key1>Key,!,
    search(L,Key).
search(t(_,Key1,R),Key):-
    search(R,Key).

```

Select one:

- ☒ a. The predicate is incorrect. It acts as an insert when the Key is not in the tree.
- ☐ b. The predicate is correct.
- ☐ c. The predicate is incorrect as it is missing the condition `Key1<Key, !,` in the 3rd clause.
- ☐ d. The predicate is incorrect, as the fact should be the second clause (search in inorder).

28. The following sequence is assumed to search for a key in an incomplete BST.
Choose the most specific answer from below:

The CORRECT answer is **D**

```
search(T,_) :- var(T),!,fail.
search(t(_,Key,_),Key) :-!.
search(t(L,Key1,_),Key) :-
    Key1>Key,!,
    search(L,Key).
search(t(_,Key1,R),Key) :-
    search(R,Key).
```

Select one:

- ☐ a. The predicate is incorrect. The first clause must be the last one.
- ☐ b. The predicate is incorrect. The second clause must be the first one.
- ☐ c. The predicate is incorrect as it is missing the condition `Key1<Key,!,` in the 4th clause.
- ☐ d. The predicate is correct.

29. The following code is assumed to collect, in order, the internal nodes with 2 children of a binary search tree. Choose the most specific answer from below:

The correct answer should be A, C is also correct as it does not specify that the tree is not empty, and this code will not work for an empty tree??

I think a + if you want them in order, the append is bad: should be append(NL, [Key|NR], NN) i think **THE MOST CORRECT ONE SHOULD BE C** (without that, A is wrong)

```

collect_2_children(t(L,_,R),NN):-
    collect_2_children(L,NL),
    collect_2_children(R,NR),
    append(NR,NL,NN).
collect_2_children(t(nil,_,R),N):-
    collect_2_children(R,N).
collect_2_children(t(L,_,nil),N):-
    collect_2_children(L,N).
collect_2_children(t(nil,_,nil),[]).

```

Select one:

- ☐ a. The predicate is incorrect as the clause with 2 recursive calls is not adding the node to the output list
- ☐ b. The predicate is correct.
- ☐ c. The predicate is incorrect, as is missing a clause on `nil`.
- ☐ d. The predicate is incorrect as the last clause should be the first one.

30. In a decreasing sorting by selection strategy (below), the `max_delete` predicate is a stable predicate which correctly deletes the maximal value from the list. If we print Out, what would be the second printed list on input In = [4,1,3,1,7]?

Correct solution is **A**: [1]. As it's backward recursive and the Out var is instantiated at the end -> smallest values will be added

```

sort(In, [M|Out]) :-
    max_delete(M, In, Int),
    sort(Int, Out).
sort([], []).

```

Select one:

- ☐ a. [1]
- ☐ b. [4,7]
- ☐ c. [1,1]
- ☐ d. [7]

```

max_delete([H|T], Max, [H|DT]) :-
    max_delete(T, Max, DT),
    H < Max,
    !.
max_delete([_Max|T], Max, T).

```

```

sort1(In, [M|Out]) :-

```

```

max_delete(In, M, Int),
sort1(Int, Out),
write(Out),
write("\n").

```

```
sort1([], []).
```

31. The predicate below should be a stable predicate to delete the max value from a list. Fill in the blanks for a correct predicate:

```

max_delete([H|T],
,
) :-
    max_delete(T, Max, DT),
    H < ,
    ! .

max_delete([H|T],
,
) .

```

32. The following sequence is assumed to insert a key in an incomplete BST. Choose the most specific answer from below.
The correct answer is: **A**, it's correct

```

ins(t(_,Key,_),Key):-!.
ins(t(L,Key1,_),Key):-
    Key1>Key,! ,
    ins(L,Key) .
ins(t(_,Key1,R),Key):-
    ins(R,Key) .

```

Select one:

- ☐ a. The predicate is correct.
- ☐ b. The predicate is incorrect, as it's missing the output argument
- ☐ c.
The predicate is incorrect as the second clause must be the first one.
- ☐ d.
The predicate is incorrect as it does not treat the case when the incomplete tree (`var(T)`) is reached).

33. The following code is assumed to collect, **in order**, the leaves of a **non-empty incomplete BST**, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:
The Correct answer is **A** probably

```

collect_leaves(t(L,Key,R),B,[Key|B]):-
    var(L),var(R),!.

collect_leaves(t(L,_,R),B,E):-
    var(R),!,
    collect_leaves(L,B,E).

collect_leaves(t(L,_,R),B,E):-|
    var(L),!,
    collect_leaves(R,B,E).

collect_leaves(t(L,Key,R),BRez,ERez):-
    collect_leaves(R,Int,ERez),
    collect_leaves(L,BRez,Int).

```

Select one:

- ☐ a.
The predicate is incorrect because the pattern of adding the leaf key is incorrect.
- ☐ b.
The predicate is incorrect as clauses 2 and 3 should occur in the opposite order.
- ☐ c. The predicate is incorrect, as is missing a first clause on `var(T)`.
- ☐ d. The predicate is incorrect as in the 4th clause the recursive calls are swapped.

34. A cut would freeze (prevent from backtracking) some nodes in the tree. However, it is NOT freezing:

So the correct answer should be **A**

Select one:

- ☒ a.
A subtree of its sibling node to its left before the cut is executed
- ☐ b. Its leftmost sibling node after the cut is executed
- ☐ c. Its parent node after the cut is executed
- ☐ d. The subtree of its leftmost sibling node after the cut is executed

The CUT (!) – effects

- ! Prevents from backtracking the following categories:
 - All left siblings
 - All subtrees of left sibling nodes
 - Parent node
- NO other node is affected:
 - Siblings on the right/their subtrees have the ability to backtrack
 - Left siblings DO backtrack BEFORE ! Is executed
 - Their subtrees also
- As everything, takes effect ONLY AFTER its (!) execution (before execution it does NOT actually exist)

3/9/2021

So the correct answer should be **A**

35. A cut NEVER affects:

Select one:

- ☐ a. Subgoals to its right
- ☐ b. Its parent node
- ☒ c. A sibling node to its left
- ☐ d. The subtree of a sibling node to its left

The correct answer is **A**

36. Consider a predicate having a recursive clause, as below:

```
p([H|T], Some_Result, End_Result):-
    combines(H, Something, New_Some_Result),
    p(T, New_Some_Result, End_Result).
```

Using this strategy, the result would have to be initialized:

Select one:

- ☒ a. When the initial call is made, in the second argument
- ☐ b. When the initial call is made, in the third argument
- ☐ c. On the stopping condition, in the second argument
- ☐ d. On the stopping condition, in the third argument

37. Consider a predicate having a recursive clause, as below:

D

```
p([H|T], Some_Result):-
    p(T, Some_Part_Result),
    combines(H, Some_Part_Result, Some_Result).
```

Using this strategy:

Select one:

- ☐ a. The result is built as recursion progresses, in Some_Result, using the value from Some_Part_Result
- ☒ b. The result is built as recursion progresses, in Some_Part_Result, using the value from Some_Result
- ☐ c. The result is built as recursion returns, in Some_Part_Result, using the value from Some_Result
- ☐ d. The result is built as recursion returns, in Some_Result, using the value from Some_Part_Result

38. Say we want a predicate which receives as input a list of integers, and produces as output a list containing their square roots, in reverse order. To obtain such a predicate (correct and complete), the recursive clause should look like:

The correct answer is **C**

Select one or more:

- ☐ a. `square([H|T], R):- NH is sqrt(H), square(T,[HN|R]).`
- ☐ b. `square([H|T], [NH|R]):- NH is sqrt(H), square(T,R).`
- ☐ c. `square([H|T], R, R):- NH is sqrt(H), square(T,[HN|R],R).`
- ☐ d. `square([H|T], R):- square(T,R).`

39. I want to write a predicate which finds the sum of all leaf nodes in a BST. Complete the partial implementation so that you get a correct predicate:

```
sumLeaf(nil, 0).  
sumLeaf(t(K, nil, nil), K).  
sumLeaf(t(K,L,R), Sum):-  
    sumLeaf(L, SL),  
    sumLeaf(R, SR),  
    Sum is SL + SR.
```

40. Given the compound term `n(c(A,B), v, m(1, [2,3]))`, which of the following terms it CANNOT unify with, and why?
The correct answer is **B**

Select one:

- ☐ a. $n(A,B,C)$, because not all three arguments can be variable
- ☐ b. $N(c(a,B), v, m(1,[2,3]))$, because the name of the structure has to be constant
- ☐ c. $N(c(a,B), v, m(1,[H|T]))$, because the third arguments do not unify
- ☐ d. $n(c(a,B), v, m(1,[2,B]))$, because the unifications involving B are in contradiction

41. The predicate below deletes one element from a list. It deletes deterministically because:

The correct answer is **C**

```
delete(H, [H|T], T) :- ! .  
  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .
```

Select one:

- ☐ a. it has just one recursive call
- ☐ b. it has no fact on the empty list
- ☐ c. it has a cut in the non-recursive clause
- ☐ d. it has no cut in the recursive clause

*_

42. The predicate below concatenates two lists. To have a nondeterministic call, we need to:

The possible correct answer is **B** (it gives us the permutations)


```
append([H|T], L, [H|R]) :-
    append(T, L, R) .

append([], L, L) .
```

Select one:

- ☐ a. change the order of the clauses
- ☐ b. make the initial call with the first 2 arguments free variables
- ☐ c. add a clause to decompose the second argument
- ☐ d. make the initial call with all 3 arguments free variables

43. The predicate below is intending to search for a given element in an **incomplete** Binary Search Tree (ended in a variable). To do so, we need to do the following:
The correct answer is **C**

```
search_iBST(K, t(_, K, _)) :- ! .
search_iBST(SK, t(L, K, _)) :-
    SK < K, !,
    search_iBST(SK, L) .
search_iBST(SK, t(_, _, R)) :-
    search_iBST(SK, R) .
```

Select one:

- ☐ a.
add the following clause `search_iBST(_, V) :- var(V), !, mandatory as the first clause.`
- ☐ b. add the following clause `search_iBST(_, V) :- var(V), !, fail.` anywhere in the code (first, second, third or fourth clause)
- ☐ c. add the following clause `search_iBST(_, V) :- var(V), !, fail.` mandatory as the first clause.
- ☐ d. add some clause for nil.

44. The predicate below has to verify if the first argument (an element) is present in the second argument. The following is true about the correctness of the predicate:
The correct answer is **C**: even if the member is present, it will backtrack and fail in the end.

fucking mind games, depends if you want it deterministic or not...I would say **D**
It's non deterministic, because there's no cut at the first clause, so it will backtrack

```

member(H, [H|_]) .
member(X, [_|T]) :-
    member(X, T) .
member(_, []) :-!, fail.

```

Select one:

- ☐ a.
it is incorrect, because it fails when reaching the empty list.
- ☐ b. it is correct and all clauses are necessary.
- ☐ c. it is incorrect, because it always fails.
- ☐ d. it is correct, but the last clause is not necessary.

45. The predicate below implements bubble sort (where append/3 has the regular, correct implementation). How many calls does the second append make to get the sorted array for the input [2,1,5,4,3]:

The correct answer is **A = 4**

```

bubble_sort(In, Out) :-
    append(X, [A, B|Y], In) ,
    A>B, !,
    append(X, [B, A|Y], Int) ,
    bubble_sort(Int, Out) .
bubble_sort(In, In) .

```

Select one:

- ☐ a. 4
- ☐ b. 2
- ☐ c. 3
- ☐ d. 5

46. The predicate swap can be used for bubble sort. The F variable:
I think it is **D**.


```

swap ([A, B|T], [B|R], F) :-
    A>B, !,
    F=y,
    swap ([A|T], R, F) .
swap ([A, B|T], [A|R], F) :-
    swap ([B|T], R, F) .

swap ([A], [A], n) :- ! .
swap ([A], [A], y) :- ! .

```

Select one:

- ☐ a. must be instantiated in the call, either to *y* or *n*
- ☐ b. gets instantiated on one of the two stopping conditions
- ☐ c. it gets re-instantiated whenever the first clause is reached during the execution
- ☐ d. gets instantiated to *y* in the first clause, then maintains its value throughout the execution

47. To make the predicate `select_sort/3` stable, we need to:

I think it is stable - `max_delete` always deletes the first occurrence of `max`, so the order will be maintained i guess

```

select_sort(In, Acc, Rez) :-
    max_delete(In, In_without_Max, Max),
    select_sort(In_without_Max, [Max|Acc], Rez) .
select_sort([], L, L) .

```

```

max_delete([H|T], [H|R], M) :-
    max_delete(T, R, M),
    M>H, ! .
max_delete([H|T], T, H) .

```

Select one:

- ☐ a. nothing, it is stable as it is
- ☐ b. swap the order of `max_delete(In, In_without_Max, Max)` and `select_sort(In_without_Max, [Max|Acc], Rez)`
- ☐ c. change the inequality into a nonstrict one
- ☐ d. add a clause for the empty list in `max_delete` predicate

48. To make the predicate `ins_sort/2` stable, we need to:

I think d, cause it goes through the list and inserts elements in the correct position, so if we add $\leq$, then it will insert the later elements after the earlier equal ones

```
ins([A|T], B, [A|R]) :-
```

```
    A < B, !,
```

```
    ins(T, B, R) .
```

```
ins(L, B, [B|L]) .
```

```
ins_sort([A|T], Out) :-
```

```
    ins_sort(T, Int),
```

```
    ins(Int, A, Out) .
```

```
ins_sort([], []).
```

Select one:

- ☐ a. nothing, it is stable as it is
- ☐ b. add a clause for the empty list in `ins/3` predicate
- ☐ c. swap the order of `ins_sort(T, Int)` and `ins(Int, A, Out)`
- ☐ d. change the strict inequality into a nonstrict one

49. The code below deletes the minimum element in a list. We executed it on list L, which we know to look as $L=[6, \dots]$. Given that, according to the specific write/writeln statements, the execution displays what you see below, how did the initial list L look like? **[6,1,2,3,1,3,4]**?

```

190 ?- del_min([6,1,2,3,1,3,4], R, M).
[] 4
[4] 3
[3,4] 1
[3,4] 1
[3,3,4] 1
[2,3,3,4] 1
[1,2,3,3,4] 1
R = [6, 1, 2, 3, 3, 4],
M = 1.

```

```
del_min([H|T], [H|R], M):-
```

```

    del_min(T,R,M),
    M=<H, !,
    write(R), write(' '), writeln(M).
del_min([H|T], T, H):-
    write(T), write(' '), writeln(H).

```

```

del_min([H|T],[H|R],M):-
    del_min(T,R,M),
    M=<H,!,
    write(R), write(' '),writeln(M).
del_min([H|T],T,H):-
    write(T), write(' '),writeln(H).

?-L=[6,...], del_min(L,New_List,Min).

[]4
[4]3
[3,4]1
[3,4]1
[3,3,4]1
[2,3,3,4]1
[1,2,3,3,4] 1

```

50. If we want a decreasing sorting by selection strategy, where MAX and DELETE are correct predicates, which condition must hold true(the most specific one):

D is correct too i think

```

sort(In,[M|Out]):-
    max(In,M),
    delete(M,In,Int),
    sort(Int,Out).

sort([],[]).

```

Select one:

- ☐ a. min needs to use a nonstrict inequality
- ☐ b. max needs to choose the first occurrence of the largest value from the list
- ☒ c. delete needs to delete the first occurrence of the value from the list
- ☐ d. min needs to be implemented with backward recursion

51. The predicate below should decreasingly sort by selection, using min and delete correct predicates. Fill the blanks:

```

wrapper(In, Out) :-
    sort(In, [], Out).

sort(In, Acc, [Acc|Int]) :-
    min(In, M),
    delete(M, In, Int),
    NA = [],
    sort(Int, Acc, Out).
sort([], Out, Out).

```

wrapper(In, Out):-
 sorting(In, [], Out).

sorting(In, Acc, **Out**):-
 min(In, M),
 delete(In, M, Int),
 NA = [M|Acc],
 sorting(Int, NA, Out).
 sorting([], Out, Out).

52. The following code is assumed to collect, in order, the internal node with 2 children of an incomplete binary search tree:

```

collect_2_children(t(L,_,R),N):-var(L),!,
    collect_2_children(R,N).
collect_2_children(t(L,_,R),N):-var(R),!,
    collect_2_children(L,N).
collect_2_children(t(L,Key,R),NN):-
    collect_2_children(R,NR),
    collect_2_children(L,NL),
    append(NL, [Key|NR], NN).
collect_2_children(t(L,_,R),[]) :-
    var(L), var(R), !.

```

Select one:

- ☐ a. The predicate is incorrect as the clause right subtree is processed before the left one.
- ☒ b. The predicate is incorrect as the last clause should be the first one.
- ☐ c. The predicate is incorrect, as is missing a clause on var(T).
- ☐ d. The predicate is correct.

53. The following code is assumed to collect, in order, the internal nodes (non leaves) of an incomplete BST, using difference lists:

The following code is assumed to collect, in order, the internal nodes (non leaves) of an incomplete BST, using difference lists (argument 2 is the beginning and argument 3 is the end of the difference list). Choose the most specific answer from below:

```
collect_internal(t(L,_,R),B,B):-
    var(L),var(R),!.

collect_internal(t(L,Key,R),B,[Key|E]):-
    var(R),!,
    collect_internal(L,B,E).

collect_internal(t(L,Key,R),[Key|B],E):-
    var(L),!,
    collect_internal(R,B,E).

collect_internal(t(L,Key,R),BRez,ERez):-
    collect_internal(R,Int,ERez),
    collect_internal(L,BRez,[Key|Int]).
```

Select one:

- ☐ a.
The predicate is incorrect because the pattern of adding the `Key` in the third clause (on the right list (`R`)) `[Key|B]` must occur in the recursive call.
- ☐ b. The predicate is incorrect, as is missing a first clause on `var(T)`.
- ☐ c. The predicate is incorrect as in the 4th clause the recursive calls are swapped.
- ☒ d.
The predicate is incorrect because the pattern of adding the `Key` in the second clause (on the left list (`L`)) `[Key|E]` must occur in the recursive call.

54. The following matching: `[A,b,C]` with `[1,2,3,4]` fails because:

Select one:

- ☐ a. The first list has 3 elements while the second one 4 elements
- ☐ b.
The first list has letters while the second one numbers
- ☐ c. The matching succeeds with instantiation: `A=1, C=[3,4]`
- ☒ d.
Both `b` and `2` are constants (and different)

55. The recursive clause of some predicate is designed with forward recursion because:


```
p([H|T], Some_Result):-  
    combines(H, Something, New_Some_Result),  
    p(T, New_Some_Result).
```

Select one:

- ☐ a. No other operation is performed after the recursive call
- ☐ b. Because Something is a hidden variable
- ☒ c.
It processes the current element before entering in recursion
- ☐ d. It is NOT forward recursion as it only has 2 arguments

56. The recursive clause below is designed with backward recursion because:

```
p([H|T],Some_Result):-  
    p(T, Some_Part_Result),  
    combines(H, Some_Part_Result,Some_Result).
```

Select one:

- ☐ a. It is NOT backward recursion as combines has 3 arguments
- ☒ b.
It processes the current element after ending the recursion
- ☐ c. No other operation is performed before the recursive call
- ☐ d. It only has 2 arguments, hence, cannot be forward

57. The following matching: $f(A,g(B),4)$ with $f(a,g,4)$ would fail because:

Select one:

- ☐ a. 4 is the 3rd argument
- ☐ b.
A should be lower case
- ☒ c. Because of the second argument ($g(B)$ is an expression while g is a constant)
- ☐ d. The matching actually succeeds

58. Which of the following clauses will the query: ?- process(a,_b,c). NOT match?
_b - variable

Select one:

- ☐ a. process(A,B,c):-
- ☐ b. process(A,B,C):-
- ☒ c. process(a, d, c):-
- ☐ d. process(a,D,C):-

59. Consider a predicate having a recursive clause, as below:

```
p([H|T], Some_Result):-  
    p(T, Some_Part_Result),  
    combines(H, Some_Part_Result, Some_Result).
```

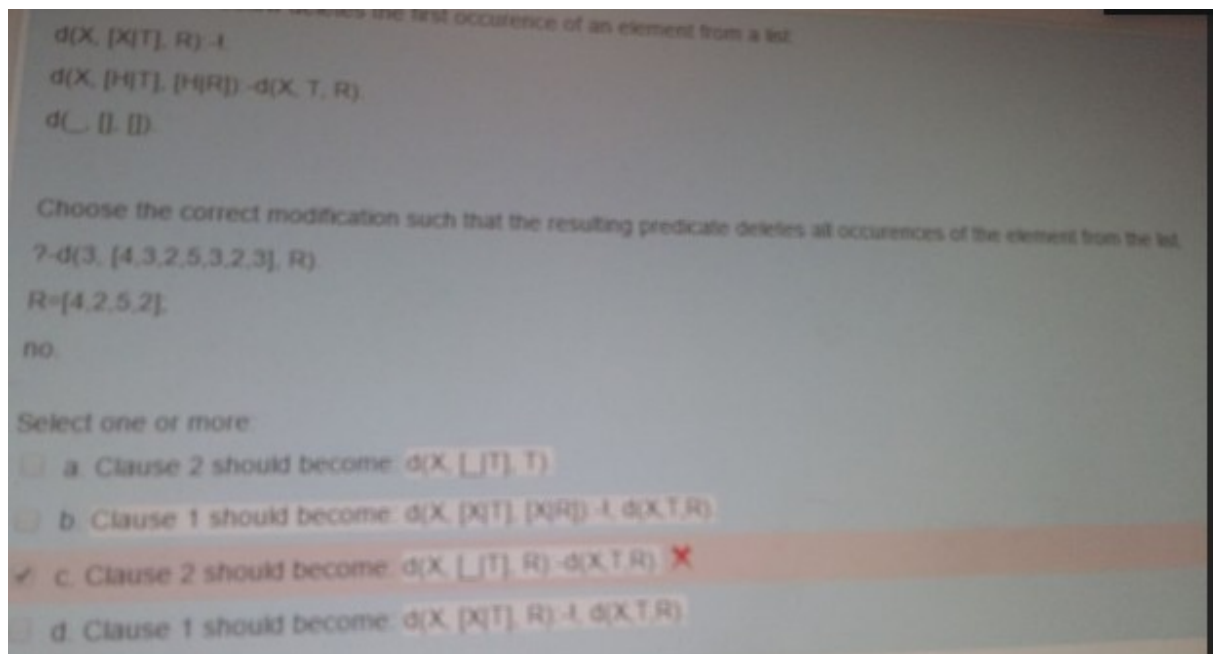
On the stopping condition:

Select one:

- ☐ a. The value of the second argument doesn't matter
- ☒ b. The result has to be initialized
- ☐ c. We obtain the final result
- ☐ d. The result has to be made available at the top level (call level)

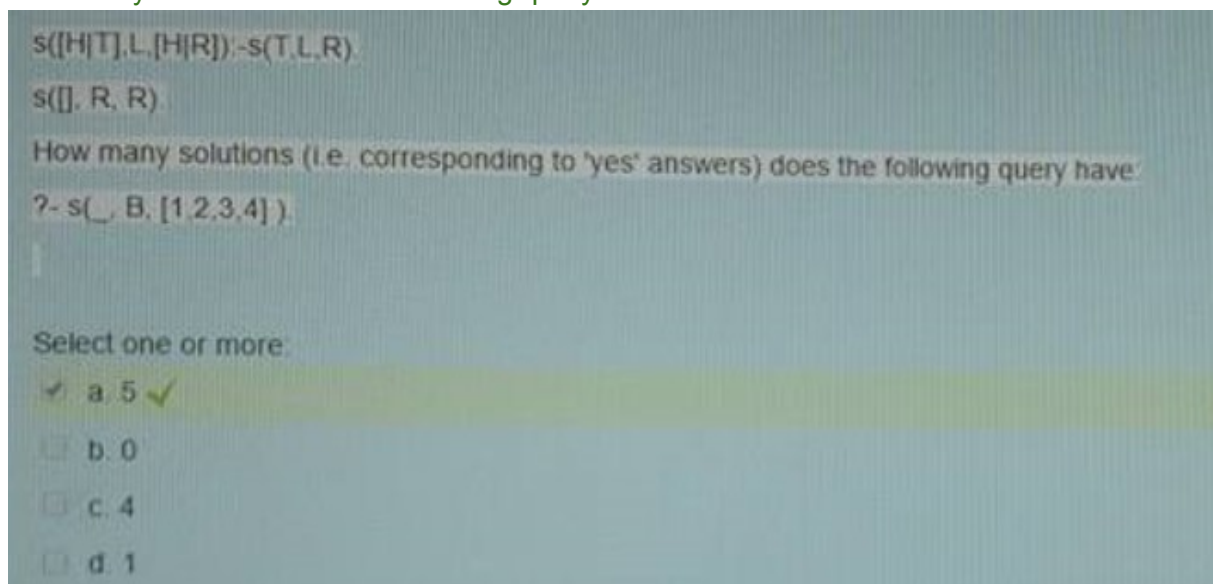
60. The predicate below deletes the first occurrence of an element from a list.
Choose the correct modification such that the resulting predicate deletes all occurrences of the element from the list.

The correct answer is **D**



65. Given the predicate:

How many solutions does the following query have



66. The following predicate performs the union of two sets (represented as lists, member is considered known and implemented)

```

u([H|T],L,R) :- member(H,L),!,u(T,L,R).
u([H|T],L,[H|R]) :- u(T,L,R).
u([],L,L).

```

Select from the alternatives below the one which correctly modifies the predicate such that it performs the intersection of two sets, instead of the union.

Select one or more:

- ☐ a. Change clause 1 head to: `u([H|T],L,[H|R]) :-`
- ☐ b. Change clause 3 to: `u([],_,[])`
- ☒ c. Change clause 1 head to: `u([H|T],L,[H|R]) :-`, the head of clause 2 to `u([H|T],L,R)` and change clause 3 to: `u([],_,[])` ✓
- ☐ d. Change clause 1 body to: `! member(H,L), !, u(T,L,R).`

Activate

67. I want to write a predicate which replaces the first (and only the first) occurrence of Old with element New, in a list. Help me complete the predicate below to achieve this task.

```

replace(Old, New, [Old|T]) [New|T] ✓, ! ✓.
replace(Old, New, [H|T]) [H|R] ✓, -replace_all(Old, New T ✓, R).
replace(____, ____ ✓, ____ ✓).

```

68. Given the predicate below. What is the answer to the query?

```

f([H|T], Acc, R) :- f(T, [H|Acc], R).
f([], R, R).

```

What is the answer to the query:
?- f([1,2,3],[4,5],R).

Select one or more:

- ☐ a. yes, R=[4,5,1,2,3]
- ☐ b. yes, R=[1,2,3,4,5]
- ☒ c. yes, R=[3,2,1,4,5] ✓
- ☐ d. yes, R=[4,5,3,2,1]
- ☐ e. no

69. The following predicate should insert an element in an ordered list. Fill in the gaps to complete its specification.

I think the first text field should be `[H|R]`.

```

insert_ord(X, [H|T], R) :- R ✗ X >= ✓ H, !, insert_ord(X, T, R).

insert_ord(X, T, [X|T] ✓)

?-insert_ord(3, [1,2,3,6],R).
R=[1,2,3,3,6];
no.

```

70. The following predicate performs the intersection of two sets (represented as lists, member is considered known and implemented)
The correct answer is **B**

```

i([H|T], L, [H|R]) :- member(H, L), !, i(T, L, R).
i([H|T], L, R) :- i(T, L, R).
i([], _, []).

```

Select from the alternatives below the one which correctly modifies the predicate such that it performs the difference of two sets, instead of the intersection.

Select one or more:

- ☐ a. Change clause 3 to: `i([], L, L).`
- ☐ b. Change clause 1 body to: `\+ member(H, L), !, i(T, L, R).`
- ☒ c. Change clause 1 head to: `i([H|T], L, R) :- ...` ✗
- ☐ d. Change clause 1 head to: `i([H|T], L, R) :- ...` and clause 3 to: `i([], L, L).`

71. The following predicate should remove duplicates from a list. Is it correct? If not, choose the necessary corrections:

```
rem([H|T], R):-member(H,T),!, rem(T, R).
```

```
rem([H|T], R):-rem(T,R).
```

```
rem([],[]).
```

```
?-rem([1,2,3,2,4,1], R).
```

```
R=[3,2,4,1];
```

```
no.
```

Select one or more:

- ☐ a. Not correct, change clause 1 to: `rem([H|T], [H|R]):-member(H,T), rem(T, R).`
- ☒ b. Not correct, change clause 2 head to: `rem([H|T], [H|R]):-` ✓
- ☐ c. It is correct.
- ☐ d. Not correct, change clause 1 head to: `rem([H|T], [H|R]):-`

QUIZ 2

71. I want to write a predicate which searches for an element in an incomplete list.

Choose the correct option to complete the code below:

```
search_il(X, [_|T]):- search_il(X, T).
```

Select one or more:

- ☐ a. Add as first two clauses:
`search_il(X, L):-var(L),!.`
- ☐ b. Add as first clause: `search_il(X, L):-var(L), !, fail.`
- ☐ c. Add as first clause: `search_il(X, [_|_]):-!.`
- ☒ d.

Add as first two clauses:

```
search_il(X, L):-var(L),!,fail.
```

```
search_il(X, [_|_]).
```



72. The following predicate searches for a certain key in an incomplete BST:


```
search_it(_, T):-var(T), !, fail.
```

```
search_it(Key, t(Key, _, _)):-!. 
```

```
search_it(Key, t(K, L, R)):-Key < K, !, search_it(Key, L).
```

```
search_it(Key, t(_, _, R)):-search_it(Key, R).
```

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

Select one or more:

- ☐ a. Change the first clause to: `search_it(_, T):-var(T), !.`
- ☐ b. Remove the first clause and change the second clause to: `search_it(Key, T):-var(T), T=t(Key,_,_).`
- ☒ c. Remove the first clause. ✓
- ☐ d. Change the first clause to: `search_it(Key, T):-var(T), !, T=t(Key,_,_),fail.`

73. Completați predicatul astfel încât să se insereze un element la sfârșitul listei incomplete: `insert_il`

```
insert_il(X, [X]) ✓
insert_il(X, [T]) ✓ :- insert_il(X, T) ✓
```

74. The following predicate checks if an element exists in a complete list

```
m(X,[X|_]):-!. 
```

```
m(X,[_|T]):-m(X,T).
```

Modificați predicatul astfel încât să verifice dacă un element există într-o listă incompletă.

Select one or more:

- ☒ a. Adaugă ca primă clauză: `m(_,L):-var(L), !, fail.` ✓
- ☐ b. Adaugă ca primă clauză: `m(_,L):-var(L), !.`
- ☐ c. Adaugă ca primă clauză: `m(_,L):-var(L), fail.`
- ☐ d. Adaugă ca primă clauză: `m(_,L):-var(L), !, fail.` și șterge "!" din clauza doi.

75. Complete the following predicate to compute the height of an incomplete binary tree:

```
height(T, 0):-var(T) ✓, [ ] ✓.
height(t(L,R), H):-height(L, HL) ✓, height(R, HR) ✓, max(HL, HR) ✓, H is HH ✓ +1.
? - height(t(4,t(2,_,_)), R).
R = 2 ;
false
```

76. The following predicate searches for a certain key in a complete BST:

```

search_it(Key, t(Key, _, _)):-!.
search_it(Key, t(K, L, R)):-Key < K, !, search_it(Key, L).
search_it(Key, t(_, _, R)):-search_it(Key, R).

```

How do we have to alter the predicate such that it performs insertion, instead of search, in an incomplete BST?

Select one or more:

- ☐ a. Change the first clause to: `search_it(Key, T):-var(Key),!,T=t(Key,_,_).`
- ☒ b. Do nothing. ✓
- ☐ c. Change the first clause to: `search_it(_, T):-var(T), !`
- ☐ d. Add, as first clause: `search_it(Key, T):-var(T), !, T=t(Key,_,_),fail.`

77. I have the predicate below:
How many yes answers do I get?

```

s(_, L):-var(L), !, fail.
s(X, [X|_]):-!.
s(X, [_|T]):-s(X, T).

```

and the query:
?- X=3, s(X,[3,3,3|_]).

How many yes answers do I get?

Select one or more:

- ☒ a. 1 ✓
- ☐ b. 4
- ☐ c. infinite number of yes answers
- ☐ d. 0
- ☐ e. 3

78. Complete the predicate to flatten an incomplete list:
It should be []

```

flatten_il(L, ) :- !.
flatten_il([H|T], ) :- !atomic(H), , flatten_il(T, ).
flatten_il([H|T], ) :- flatten_il(H, ) , flatten_il(T, ) , append(R1, R2, ) .

```

Ex apel:
?- flatten_il([1, [2|_]|_], R).
R = [1,2];
no

79. The following predicate verifies if an element exists in a complete list:
Modify it so that it verifies if the element exists in a deep list and make it not-deterministic:

```

m(X,[_|_]) :- !.
m(X,[_|T]) :- m(X,T).

```

Modificați predicatul astfel încât să verifice dacă un element există într-o listă adâncă și să fie nedeterminist.

Select one or more:

- ☒ a. Adaugă clauza: $m(X,[_|_]) :- m(X,H).$ și șterge "!" din prima clauză ✓
- ☐ b. Schimba a doua clauză în: $m(X,[_|T]) :- m(X,H), !, m(X,T).$
- ☐ c. Schimba a doua clauză în: $m(X,[_|T]) :- m(X,H), m(X,T).$ și șterge "!" din prima clauză
- ☐ d. Adaugă între prima și a doua clauză: $m(X,[_|_]) :- m(X,H).$

80. Complete the following predicate such that it computes the depth of a deep list:
It's probably $D1+1$ (without blanks)

```

d([],1).
d([H|T],D) :- !atomic(H), , d(T, ) , ) .
d([H|T],D) :- d(H,D1), d(T,D2), D1 is , max(, ) , D).

```

81. The following predicate has to insert an element at the end of an incomplete list (if it is already in the list, it doesn't get added). Choose the right variant:

insert_il(X, [_|T]) :- insert_il(X, T).

D?

Select one or more:

- ☐ a. Add, as first clause: `insert_il(X, L):-var(L), !, fail.`
- ☒ b. Add, as first two clauses:
`insert_il(X, L):-var(L), L=[X|_].`
`insert_il(X, [X|_]).` ✗
- ☐ c. Add, as first clause: `insert_il(X, [X|_]).`
- ☐ d. Add, as first clause: `insert_il(X, [X|_):-!`

QUIZ 3

82. Completati predicatul urmator astfel incat sa gaseasca un drum intr-un graf orientat cu lungimea mai **mare** decat M.

I think it should be **[Z|PP]**

```
p(X,Y,M,P):- p(X,Y,M,[X],P).  
p(X,X,M, PP,PP):-  
    length(PP, L),  
    L > M.  
p(X,Y,M, PP, FP):-  
    edge(X,Z),  
    not(member(Z, PP)),  
    p(Z, Y, M, PP, FP).
```

83. Completati predicatul urmator astfel incat sa gaseasca un drum intr-un graf orientat cu lungimea mai **mica** decat M.

I think it should be **[Z|PP]**

p(X,Y,M,P):- p(X,Y,M,[X],P).

```

p(X,Y,M,P):- p(X,Y,M,[X],P).
p(X,  ✓, M, PP, PP):-
    length( ✓, L),
     ✓ <  ✓.
p(X,  ✓, M, PP, FP):-
    edge(X,Z),
    not(member( ✓, PP)),
    p( ✓, Y, M,  ✗, FP).

```

84. Complete the predicate below such that it finds a path in a directed graph of length greater than M: (SEE ABOVE)

```

p(X,Y,M,P):- p(X,Y,M,[X],P).
p(X,X,M, PP,PP):-
    length(, L),
    .
p(X,Y,M, PP, FP):-
    edge(X,Z),
    \+(member(, PP)),
    p(, Y, M, , FP).

```

85. Alegeti intrebarea care adauga elementul "a" la sfarsitul unei liste diferenta ce contine elementele 1, 2 si 3.

q(X, SL, [X|EL], SL, EL).

$q(X, SL, [X|EL], SL, EL).$

Select one or more:

- ☐ a. $?-q(a, [1,2,3], [], RS, RE).$
- ☐ b. $?-q(a, [1,2,3|_], _, RS, RE).$
- ☐ c. $?-q(a, [1,2,3|_], _, [], RE).$
- ☒ d. $?-q(a, [1,2,3|L], L, RS, []).$ ✓

86. Se da predicatul de mai jos:

$q(X, SL, [X|EL], SL, EL).$

$q(X, SL, [X|EL], SL, EL).$

Care este răspunsul la următoarea întrebare:

$?-q(3, [1,2,3|L], L, SL, []).$

Select one or more:

- ☐ a. false
- ☐ b. $SL = [1,2,3,3]$
- ☒ c. $L = [3], SL = [1,2,3,3]$ ✓
- ☐ d. $L = [3], SL = [1,2,3]$

87. Completați predicatul de mai jos astfel încât să traverseze arborele în **post-ordine** folosind liste diferențiale.

I guess it's **[K|LE]**

$p_dl(nil, R, \boxed{R}).$
 $p_dl(t(K,L,R), \boxed{LS}, LE):-p_dl(L, LS, \boxed{ML}), p_dl(R, \boxed{R}, ML, \boxed{LE}).$

88. Completați predicatul de mai jos astfel încât să traverseze arborele în **pre-ordine** folosind liste diferențiale.

I guess it's **[K|LS]**


```

p_dl(nil, T,  ✓).
p_dl(t(K,L,R),  ✗, LE):-p_dl(L, LS,  ✓), p_dl(R,  ✓, ML,  ✓).

```

89. Complete the predicate below to perform the **inorder** traversal, using difference lists.

```

inord_dl(nil, L,  ✓).
inord_dl(t(K,L,R), LS, LE):-inord_dl(L,  ✓,  ✓), inord_dl(R,  ✓, ML,  ✓).

```

90. Predicatul de mai jos trebuie sa inverseze o lista folosind liste diferenta:

```

reverse([],_,_).
reverse([H|T],Rs,Re):-reverse(T,Rs,[H|Re]).

```

Ce trebuie modificat?

Select one or more:

- ☐ a. Modifică clauza 2 în: `reverse([H|T],[H|Rs],Re):-reverse(T,Rs,Re).` și clauza 1 în: `reverse([],L,L).`
- ☐ b. Modifică clauza 2 în: `reverse([H|T],Rs,[H|Re]):-reverse(T,Rs,Re).` și clauza 1 în: `reverse([],L,L).`
- ☐ c. Schimbă ordinea clauzelor.
- ☒ d. Modifică clauza 1 în: `reverse([],L,L).` ✓

```
reverse([],_,_).
```

```
reverse([H|T],[H|Rs],Re):-reverse(T,Rs,Re).
```

Ce trebuie modificat?

Select one or more:

- ☐ a. Schimbă ordinea clauzelor.
- ☐ b. Modifică clauza 2 în: `reverse([H|T],Rs,[H|Re]):-reverse(T,Rs,Re).` și clauza 1 în: `reverse([],L,L).`
- ☒ c. Modifică clauza 2 în: `reverse([H|T],Rs,Re):-reverse(T,Rs,[H|Re]).` și clauza 1 în: `reverse([],L,L).` ✓
- ☐ d. Modifică clauza 1 în: `reverse([],L,L).`

91. Se da urmatoare secventa de intrebari:

The correct answer is **D**: `n(1), n(2)`

?- assertz(n(1)), assertz(n(2)), asserta(n(3)).

?- retract(n(X)),!, fail.

Cum va arăta baza de cunoștințe după execuția celor 2 întrebări?

Select one or more:

- ☒ a. Nimic ✗
- ☐ b. `n(2).`
`n(3).`
- ☐ c. `n(2).`
`n(1).`
- ☐ d. `n(1).`
`n(2).`

92. Se da urmatoare secventa de intrebari:

The correct answer is **Nothing** (no cut)

?- assertz(n(1)), assertz(n(2)), asserta(n(3)).

?- retract(n(X)), fail.

Cum va arăta baza de cunoștințe după execuția celor 2 întrebări?

Select one or more:

☐ a. n(2).

n(3).

☒ b. n(1).

n(2). ✗

93. Complete the following predicate to make it find cycles in a graph containing the node X:

```
c(X, [X | P]) :- arc(X, Y), c(Y, X, [Y], P).  
c(X, Y, PP, FP) :-  
    arc(X, Z),  
    not(member(Z, PP)),  
    c(Z, Y, [Z|PP], FP).  
c(X, X, PP, P) :- reverse(PP, P).
```

94. Predicatele de mai jos trebuie să găsească primul **drum simplu** (și doar primul) între 2 noduri din graf (reprezentat cu predicate edge).

By crossing out the incorrect ones, the correct answer should be:

Modifica clauze 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z, Path)),!,path(Z,Y,[Z|Path],Res),!. ????

path(X,X,P,P).

path(X,X,P,P).
path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)), path(Z,Y, [Z|Path],Res).

Ce trebuie modificat?

Select one or more:

Raspunsul cel mai lung e cel mai corect ca la Conquistador

- ☒ a. Modifică clauza 1 în: path(X,X,P,P):-!. și clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!, path(Z,Y, [Z|Path],Res).
- ☐ b. Modifică clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!, path(Z,Y, [Z|Path],Res).
- ☐ c. Modifică clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)), path(Z,Y, [Z|Path],Res), !.
- ☒ d. Modifică clauza 1 în: path(X,X,P,P):-!. **✗**

path(X,X,P,P).
path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)), path(Z,Y, [Z|Path],Res).

Ce trebuie modificat?

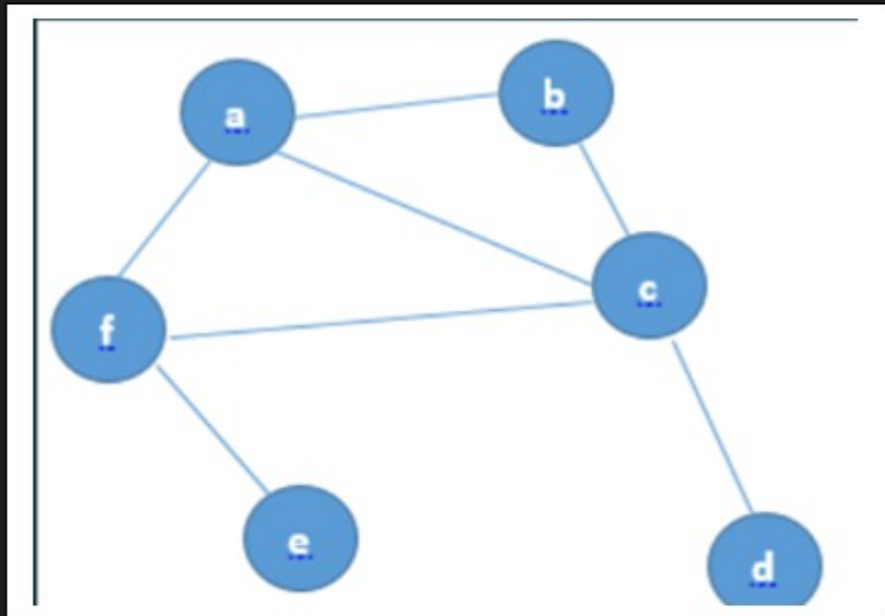
Select one or more:

- ☐ a. Modifică clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!, path(Z,Y, [Z|Path],Res).
- ☒ b. Modifică clauza 1 în: path(X,X,P,P):-!. și clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)),!, path(Z,Y, [Z|Path],Res). **✗**
- ☐ c. Modifică clauza 1 în: path(X,X,P,P):-!.
- ☐ d. Modifică clauza 2 în: path(X,Y,Path,Res):-edge(X,Z), not(member(Z,Path)), path(Z,Y, [Z|Path],Res), !.

```

1066 path(X,X,P,P).
1067 path(X,Y,Path,Res) :-
1068     edge(X,Z),
1069     not(member(Z,Path)),
1070     path(Z,Y,[Z|Path], Res), !.

```



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL COMM

```

88 ?- path(a,d,[a],P).
P = [d, c, b, a].

```

So I think the correct answer is: Modifica clauza 2 in `path(X,Y,Path,Res) :- edge(X,Z), not(member(Z,Path)),path(Z,Y,[Z|Path], Res), !.`

95. Correct the predicate below such that it can convert both complete and incomplete lists into a diff list:

Correct answer is **B**

```
convert([], E, E).
```

```
convert([H|T], S, E):- convert(T, [H|S], E).
```

Select one or more:

- ☐ a. change first clause to `convert(L, E, E):- var(L), !.`
and second clause to `convert([H|T], S, E):- convert(T, S, [H|E]).`
- ☐ b. add as first clause `convert(L, E, E):- var(L), !.`

and correct last clause to `convert([H|T], [H|S], E):- convert(T, S, E).`

☒ c. add as first clause `convert(L, E, E):- var(L), !.` and correct second clause to `convert([H|T], S, E):- convert(T, S, [H|E]).`



☐ d. change first clause to `convert(L, E, E):- var(L), !.`

```
convert(L,E,E):- var(L), !.
```

```
convert([],E,E).
```

```
convert([H|T], [H|S] ,E):- convert(T, S, E).
```

96. You are given the predicate below and the example graph on the right (undirected, edge-clause form, nodes appear in lexicographic order):

I think it is **A**

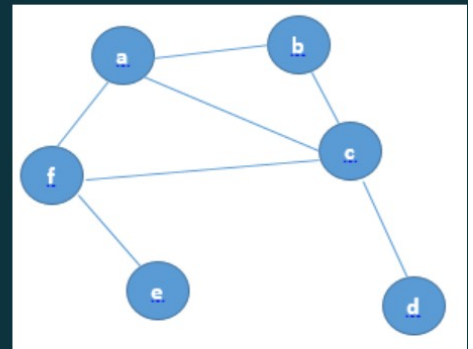
```
d(X):- assertz(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).
```

What will the predicate base look like after the execution of:
?- d(a).

Select one or more:

- ☐ a. vert(a).
vert(b).
vert(c).
vert(d).
vert(f).
vert(e).
- ☐ b. vert(a).
vert(b).
vert(c).

vert(d).
- ☐ c. vert(e).
vert(f).
vert(d).
vert(c).



```
edge(a,b). edge(a,c). edge(a,f). edge(b,a). edge(b,c). edge(c,a). edge(c,d). edge(c,f).  
edge(d,c). edge(e,f). edge(f,a). edge(f,c). edge(f,e).
```

```
d(X):- assertz(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).
```

97. You are given the predicate below and the example graph on the right (undirected, edge-clause form, nodes appear in lexicographic order):

`d(X):- asserta(vert(X)), edge(X,Y), \+(vert(Y)), d(Y).`

What will the predicate base look like after the execution of:

?- d(a).

Select one or more:

☐ a. `vert(a).`

`vert(b).`

`vert(c).`

`vert(d).`

`vert(f).`

`vert(e).`

☒ b. `vert(d).`

`vert(c).`

`vert(b).`

`vert(a).` ✗

☒ c. `vert(e).`

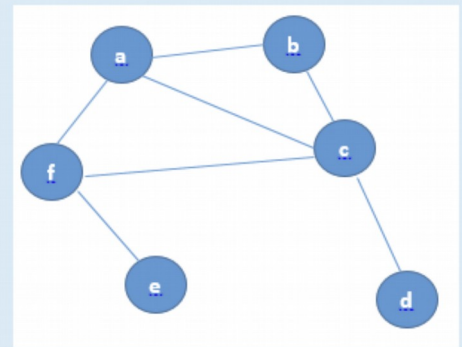
`vert(f).`

`vert(d).`

`vert(c).`

`vert(b).`

`vert(a).` ✓



98. In an undirected graph represented by its edges (e.g. `is_edge(a,b).`), the predicate checking from below is assumed to check that each vertex in the graph is possessing a certain property which is soundly checked by the `verify(Vertex)` predicate. Assuming that `verify` is correctly implemented, choose the most specific answer from below

```

checking:- is_edge(X,_),
           not(v(X)),
           assertz(v(X)),
           verify(X),
           fail.
checking.

```

Select one:

☐ a.

The predicate is correct if `verify` is negated (`not(verify(X))`)

☐ b.

The predicate is correct if `assertz` is replaced by `asserta`.

☐ c.

The predicate is correct if `verify` is negated (`not(verify(X))`) and a cut is added just after it (`not(verify(X)), !,.`)

☐ d.

The predicate is correct if a cut is added after `verify` like (`verify(X), !,.`)

99. In an undirected graph represented by its edges, the predicate constructing from

below is assumed to build an additional knowledge base of good nodes containing all nodes of the graphs for which a certain property holds true:

I think **C** is correct

```
constructing:- is_edge(X,_),
               not(v(X)),
               assertz(v(X)),
               not(verify(X)),
               assertz(good(X)),
               fail.

constructing.
```

Select one:

- ☐ a.
The predicate is correct as it is.
- ☐ b. The predicate is correct if the ! is added after not(verify(X)).
- ☒ c.
The predicate is correct if the predicate verify is NOT negated (not(verify(X)) replaced by verify(X)).
- ☐ d.
The predicate is correct if assertz is replaced by asserta.

Here is an example I tried:

edge(1,2).

edge(1,3).

edge(2,3).

edge(2,1).

edge(4,3).

:-dynamic v/1.

constructing :-

is_edge(X, _),

not(v(X)),

assertz(v(X)),

verify(X),

assertz(good(X)),

fail.

constructing.

verify(X):- 0 is (X mod 2).